

Some code that might be useful even if not optimal, or even correct

K. Pister, 10/23/12

```
mfc0    $k0, Cause
mfco    $k1, Status
andi    $k1, $k1, 0xfc00
and     $k0, $k0, $k1
andi    $k1, $k0, 1<<15
bne     $k1, $0, timer_ISR
andi    $k1, $k0, 1<<14
bne     $k1, $0, RTC_ISR
andi    $k1, $k0, 1<<10
bne     $k1, $0, UART_ISR
j       done
nop
```

timer_ISR: // print mm:ss every second

```
...
mfc0    $k1, Compare
la      $k0, 5*10^7
addu    $k0, $k0, $k1
mtc0    $k0, Compare
...
J       done
...
```

RTC_ISR:

```
lw      $k0, SW_RTC
nop
addi    $k0, $k0, 1
sw      $k0, SW_RTC
mfc0    $k1, Status
andi    $k1, $k1, !(1<<14)
mtc0    $k1, Status
j       done
...
```

done: // eret would be shorter

```
mfc0    $k1, Status
ori     $k1, $k1, 1
mfc0    $k0, EPC
j       $k0
mtc0    $k1, Status
```

UARTRX: // state = input character

```
//-----
```

```
r100M:
```

```
addi    $t0, $0, 0
```

```
la      $t1, 10^7
```

```
loop:
```

```
addi    $t0, $t0, 1
```

```
bne     $t0, $t1, loop
```

```
nop
```

```
//-----
```

```
lw      $t0, b
```

```
nop
```

```
addi    $t0, $t0, 1
```

```
sw      $t0, b
```

```
//-----
```

```
addi    $a0, $t0, 0
```

```
jal     addone
```

```
nop
```

```
addi    $t0, $v0, 0
```

```
#define COUNT 0x10000010
```

```
int tstart, tend;
```

```
register int a;
```

```
static volatile int b, state = 'r';
```

```
char s[100];
```

```
...
```

```
while(1) switch(state) {
```

```
    case 'r':          // register variable addi
```

```
        tstart = *COUNT;
```

```
        r100M();
```

```
        tend = *COUNT;
```

```
        sprintf(s,"r: %d",tstart-tend);
```

```
        out(s);
```

```
        break;
```

```
    case 'R':          // register variable, plusone function call
```

```
        ...
```

```
    case 'v':          // volatile variable, addi
```

```
    case 'V':          // volatile variable, plusone function call
```

```
    case 'm':          // cache miss variable? (optional)
```

```
    case 'd':          // disable timer interrupt
```

```
    case 'e':          // enable timer interrupt
```

```
    default:           // print error? (optional)
```

```
}
```

Example output might look something like this

00:01

00:02

00:03

00:04

r: 200,000,192 cycles

00:05

de <- assume a couple of seconds pass between pressing d and pressing e

00:08

r:200,000,048 cycles

00:09

v00:10 <- causes the *next* count to use a volatile variable

00:11

00:12

r: 200,000,192 cycles

00:13

00:14

00:15

00:16

00:17

00:18

v: 300,000,250 cycles

00:19